

HELIOS

Steps 016 to 025, explained

What we built, how every file and line of code works, and why we did it: Python project, tests, code checkers, commit hooks, GitHub CI, Docker, PostgreSQL

Team: Abhishek Kumar · Anuj Sharma · Indra Shikari

Mentor: Dr. Shripal Vijayvargiya · BIT Mesra, CSE

Date: 12 September 2026

Scope: Steps 016 to 025 are finished and verified. Step 026 (the config loader) has **not been started yet**; it is described at the end as the next step. All code listings in this guide are copied automatically from the real project files.

Contents

#	Section	In one line
1	The big picture	Where everything lives and how the pieces fit together
2	Words you will see	A small dictionary of every technical term used
3	Step 016: Python project	pyproject.toml, uv, uv.lock, virtual environment, libraries
4	Step 017: First test	pytest and the smoke test
5	Step 018: Code checkers	ruff (style and mistakes) and mypy (types)
6	Step 019: Commit hooks	pre-commit and gitleaks (secret blocker)
7	Step 020: GitHub CI	GitHub Actions checks every push automatically
8	Steps 021 and 022: Team	TEAM.md and laptop setup for everyone
9	Step 023: Docker and PostgreSQL	Containers, images, volumes, docker-compose.yml, .env
10	Step 024: Adminer	Looking inside the database from the browser
11	Step 025: Python talks to the database	db.py and test_db.py explained line by line
12	How a change travels	From editing a file to a green check on GitHub
13	Problems we hit and fixed	Every error so far and its fix
14	Cheat sheet	Everyday commands
15	Rules and what comes next	Team rules and step 026

1. The big picture

Before step 016 we prepared the laptop and the project folder (steps 001 to 015): freed disk space, confirmed Ubuntu 24.04 inside Windows (WSL2), set up git with an SSH key for GitHub, got Docker Desktop working, installed VS Code, decided team roles, made the GitHub repo private, created the folder structure, wrote the .gitignore rules, moved the documents into docs/, and checked the 338 MB of market data (139 of 139 archives OK).

Steps 016 to 025 turned that empty folder into a **real, checked, tested software project** that can already talk to its own database. This is how the pieces fit:

Figure 1: where everything lives

```
Your laptop (Windows 11)
|
+-- Project folder (the ONLY copy):  C:\Users\bit\OneDrive\Desktop\helios
|   seen from Ubuntu as:             /mnt/c/Users/bit/OneDrive/Desktop/helios
|
+-- Ubuntu 24.04 (WSL2) <- all developer commands run here
|   +-- uv          -> Python 3.12 + libraries in ~/.venvs/helios      (step 016)
|   +-- pytest      -> runs the tests                                (step 017)
|   +-- ruff, mypy   -> check code style, mistakes and types          (step 018)
|   +-- pre-commit   -> runs checks automatically on every commit      (step 019)
|   +-- gitleaks     -> blocks passwords and keys in commits           (step 019)
|
+-- Docker Desktop
|   +-- helios-postgres (PostgreSQL 16 database)                      (step 023)
|   +-- helios-adminer  (web page to look inside it)                  (step 024)
|
GitHub (private repo AbhishekKumar-cse/helios-trading-engine)
+-- GitHub Actions CI -> runs uv sync, ruff, mypy, pytest on every push (step 020)
```

Files created or changed in steps 016 to 025

File	Step	What it is
pyproject.toml	016+	Main settings file of the Python project
uv.lock	016+	Exact versions of every library (46 packages now)
.python-version	016	Says: use Python 3.12
research/helios/__init__.py	016	Makes helios an importable Python package
research/tests/test_smoke.py	017	First test
.pre-commit-config.yaml	019	List of checks that run before every commit
.github/workflows/ci.yml	020	GitHub CI workflow
docs/TEAM.md	021	Team rulebook
infra/docker-compose.yml	023	Recipe for the PostgreSQL + Adminer containers
.env.example	023	Settings template with a fake password (committed)
.env	023	Real settings with the real password (private, never committed)
research/helios/common/__init__.py	025	Makes common a sub-package
research/helios/common/db.py	025	Database connection code
research/tests/test_db.py	025	Database tests
.gitignore	017	Added .hypothesis/ to the ignore list

2. Words you will see

Word	Simple meaning
Terminal	A text window where you type commands instead of clicking.
WSL2 / Ubuntu	A real Linux system running inside Windows. Most programming tools and all servers use Linux, so we work there.
Python	The programming language of the HELIOS research code. We fixed version 3.12 for everyone.
Library (package)	Ready-made code someone else wrote that we reuse, for example pandas for tables.
Dependency	A library our project needs in order to run.
Virtual environment (venv)	A private box holding Python and libraries for ONE project, so projects never mix.
uv	The tool that installs Python, installs libraries, creates the venv and writes the lock file. Very fast.
Lock file (uv.lock)	An exact shopping bill of every library version, so every laptop and CI install the same thing.
TOML	Tom's Obvious, Minimal Language. A simple text format for settings files (pyproject.toml).
YAML	YAML Ain't Markup Language. Another simple settings format, based on indentation (ci.yml, docker-compose.yml).
Test	A small program that checks that our code gives the right answer.
Linter	A tool that reads code and points out mistakes and bad style without running it (ruff).
Type checker	A tool that checks we pass the right kind of value, like a number and not text (mypy).
git / commit / push	git records versions of files. A commit is one saved snapshot. Push uploads commits to GitHub.
Git hook	A small script git runs automatically at a moment, for example just before a commit.
CI (Continuous Integration)	A server that automatically checks the code on every push and shows green or red.
Docker	A tool that runs programs inside isolated boxes called containers, the same way on every computer.
Image	A frozen, downloadable template of a program (like an installer that never changes).
Container	A running copy of an image (the program actually working).
Volume	Storage owned by Docker that survives when a container is removed (like an external drive).
Database / PostgreSQL	A program that stores data in tables safely. PostgreSQL is a free, professional one.
SQL	The language used to ask a database questions, for example SELECT 1.
Environment variable	A named setting given to a program from outside the code, for example POSTGRES_USER.
.env file	A private text file holding environment variables such as the database password.
Secret	Anything that must never be public: passwords, API keys, tokens.
Checksum (SHA-256)	A fingerprint of a file. If even one byte changes, the fingerprint changes.

3. Step 016: creating the Python project

What we did

- Wrote `pyproject.toml` by hand (the project settings file).
- Pinned Python to 3.12 with `.python-version`.
- Created the package file `research/helios/__init__.py`.
- Used **uv** to download and install the libraries and to write `uv.lock`.
- Put the virtual environment **outside OneDrive**, at `~/venvs/helios` inside Ubuntu.

Why

Every later step writes Python code. Code needs a proper project: a name, a fixed Python version, a list of libraries with exact versions, and a known place for the code. Without this, each teammate would install different versions and get different results, which breaks the HELIOS rule that every result must be reproducible.

The file `pyproject.toml` (current version)

`pyproject.toml`, copied from the project

```
[project]
name = "helios"
version = "0.1.0"
description = "HELIOS: an alpha-driven quantitative trading system with fault-tolerant and low-latency trading infrastructure"
requires-python = ">=3.12"
authors = [
    { name = "Abhishek Kumar" },
    { name = "Anuj Sharma" },
    { name = "Indra Shikari" },
]
dependencies = [
    "alembic>=1.20.0",
    "duckdb>=1.5.5",
    "numpy>=2.5.3",
    "pandas>=3.0.5",
    "psycpg[binary]>=3.3.5",
    "pyarrow>=25.0.1",
    "pydantic>=2.13.5",
    "pydantic-settings>=2.15.0",
    "pyyaml>=6.0.3",
    "sqlalchemy>=2.0.52",
]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

```

# The Python package lives in research/helios (see ARCHITECTURE_AND_TECH_STACK.md §7)
[tool.hatch.build.targets.wheel]
packages = ["research/helios"]

[tool.pytest.ini_options]
testpaths = ["research/tests"]

# ---- ruff: code style + lint (step 018) ----
[tool.ruff]
line-length = 100
target-version = "py312"
src = ["research"]
# Only check real Python code, never the Python examples inside Markdown docs
extend-exclude = ["*.md"]

[tool.ruff.lint]
# E/W = pycodestyle, F = pyflakes (real mistakes), I = import sorting,
# B = bugbear (likely bugs), UP = modern Python syntax
select = ["E", "W", "F", "I", "B", "UP"]

[tool.ruff.lint.isort]
known-first-party = ["helios"]

# ---- mypy: strict type checking for the helios package (step 018) ----
[tool.mypy]

python_version = "3.12"
strict = true
mypy_path = "research"
plugins = ["pydantic.mypy"]

[dependency-groups]
dev = [
    "hypothesis>=6.168.0",
    "mypy>=2.3.1",
    "pre-commit>=4.6.2",
    "pytest>=9.1.1",
    "ruff>=0.16.7",
]

```

pyproject.toml explained part by part

Part	What it means	Why we need it
[project] name, version	The package is called helios, version 0.1.0.	Lets Python and tools identify our code.
requires-python = ">=3.12"	Python 3.12 or newer is required.	Our code uses 3.12 features; older Python must refuse to install it.
authors	The three team members.	Credit and ownership.
dependencies	Libraries the project needs to run. >= means this version or newer.	uv installs these. Exact versions live in uv.lock.
[build-system] hatchling	The tool that turns our folder into an installable package.	Needed so import helios works from anywhere.
packages = ["research/helios"]	Our code lives in research/helios.	The default would be src/, which our architecture does not use.
[tool.pytest.ini_options]	pytest looks for tests only in research/tests.	Never wastes time scanning data/ or docs/ (step 017).
[tool.ruff] / [tool.mypy]	Settings of the code checkers.	Explained in step 018.
[dependency-groups] dev	Tools for developers only (pytest, ruff, mypy, hypothesis, pre-commit).	They are needed to build and check code, not to run it.

The other small files

File	Content	Why
.python-version	3.12	Tells uv which Python to use, so everyone uses the same one.
research/helios/__init__.py	__version__ = "0.1.0" plus a one-line description	A folder with __init__.py is a Python package. Without it, import helios fails.
uv.lock	Machine-written list of exact versions and checksums of all packages.	Same versions on every laptop and in CI. Never edit by hand; uv updates it.

The virtual environment outside OneDrive

The virtual environment holds Python and all libraries: thousands of small files (about 419 MB). If it lived inside the OneDrive folder, OneDrive would try to upload every file, slow the laptop down and sometimes lock files while tools use them. So the **code stays in OneDrive**, but the **library box lives in Ubuntu** at `/home/abhi/.venvs/helios`.

uv finds that box through one setting, added to the Ubuntu start-up files `~/.bashrc` and `~/.profile`:

```
export UV_PROJECT_ENVIRONMENT="$HOME/.venvs/helios"
```

Every time you open an Ubuntu terminal, this line runs, so every `uv run ...` command automatically uses the right environment. Nobody needs to remember the path.

The libraries we installed and why HELIOS needs them

Library	Version	What it does	Why HELIOS needs it
numpy	2.5.3	Very fast maths on large arrays of numbers	Returns, volatility, Sharpe ratio, z-scores
pandas	3.0.5	Tables of data inside Python (DataFrames)	Loading price candles, time columns, features over time
pyarrow	25.0.1	Reads and writes Parquet, a fast compressed table format	Nine years of bars load in seconds (step 051)
duckdb	1.5.5	A small SQL database that runs inside Python	Querying Parquet files with SQL (step 054)
pydantic	2.13.5	Checks data has the right types	Config files are validated, wrong types give clear errors
pyyaml	6.0.3	Reads YAML files	Gates, fees, universe and splits are YAML files
pytest	9.1.1	Runs tests (dev)	Every step needs tests
hypothesis	6.168.0	Tries thousands of random inputs (dev)	Property tests: P&L always balances, order rules hold
ruff	0.16.7	Finds mistakes and formats code (dev)	Same clean style for all three of us
mypy	2.3.1	Checks types before running (dev)	Catches wrong-type bugs in trading maths
pre-commit	4.6.2	Runs checks before each commit (dev)	Blocks bad code and secrets (step 019)

Note: uv also installed helper packages that these libraries need (for example `python-dateutil` for `pandas`, `pydantic-core` for `pydantic`, `pluggy` for `pytest`). We asked for 11 libraries and got 37 packages in step 016. Downloading about 130 MB took about 38 minutes on our slow internet; installing took under one second. Downloads are cached, so this cost is paid only once.

4. Step 017: the first test

What we did and why

We wrote a tiny test and told pytest where tests live. A **smoke test** only checks that the basic machinery works, like switching a new machine on to see that no smoke comes out. It proves that pytest runs and that Python can import our helios package. Every future feature will add real tests in the same folder.

research/tests/test_smoke.py

```
"""Smoke test: proves pytest runs and the helios package is importable (step 017)."""

import helios

def test_smoke() -> None:
    assert 1 + 1 == 2
    assert helios.__version__ == "0.1.0"
```

Line	Meaning
"""Smoke test: ..."""	A docstring: a description of the file for humans.
import helios	Loads our package. If the project setup were broken, this line would fail.
def test_smoke() -> None:	pytest automatically runs every function whose name starts with test_. -> None says the function returns nothing (for mypy).
assert 1 + 1 == 2	assert means: this must be true, otherwise the test fails.
assert helios.__version__ == "0.1.0"	Checks the imported package really is ours, version 0.1.0.

We also added this to pyproject.toml so pytest only searches the tests folder:

```
[tool.pytest.ini_options]
testpaths = ["research/tests"]
```

Result when running `uv run pytest -v`:

```
research/tests/test_smoke.py::test_smoke PASSED [100%]
===== 1 passed in 2.14s =====
```

Note: Running tests creates cache folders (`.pytest_cache` and `__pycache__`). git already ignores them. We added `.hypothesis/` to `.gitignore` in advance, because the hypothesis tool creates it later.

5. Step 018: code checkers ruff and mypy

Why we need automatic checkers

Three people will write thousands of lines. Without rules, everyone formats code differently and small mistakes (an unused variable, a wrong type) slip in quietly. In trading maths a silent mistake gives wrong numbers that still look believable, which is the worst kind of bug. Checkers read the code and complain **before** anything runs.

ruff settings

```
[tool.ruff]
line-length = 100
target-version = "py312"
src = ["research"]
extend-exclude = ["*.md"]

[tool.ruff.lint]
select = ["E", "W", "F", "I", "B", "UP"]

[tool.ruff.lint.isort]
known-first-party = ["helios"]
```

Setting	Meaning	Example of what it catches or does
line-length = 100	At most 100 characters per line	Lines stay readable on any screen
target-version = "py312"	Our code is Python 3.12	Suggests modern 3.12 syntax
src = ["research"]	Where our own code lives	Import sorting knows helios is ours
extend-exclude = ["*.md"]	Never touch Markdown documents	Newer ruff wanted to reformat the Python example inside ARCHITECTURE_AND_TECH_STACK.md; documents must not be rewritten by a code tool
E, W (pycodestyle)	Basic style problems	Wrong spacing, trailing whitespace
F (pyflakes)	Real mistakes	Unused import, a name that does not exist
I (isort)	Import order	Imports sorted the same way in every file
B (bugbear)	Patterns that often cause bugs	A list used as a default argument
UP (pyupgrade)	Modern syntax	Use list[int] instead of the old List[int]
known-first-party = ["helios"]	helios is our own package	Our imports are grouped after library imports

mypy settings

```
[tool.mypy]
python_version = "3.12"
strict = true
mypy_path = "research"
plugins = ["pydantic.mypy"]
```

Setting	Meaning
strict = true	The strictest mode: every function must declare the types it takes and returns, and unsafe shortcuts are errors.
mypy_path = "research"	Tells mypy where the helios package is.
plugins = ["pydantic.mypy"]	Teaches mypy how pydantic models work, so settings classes (step 025) are checked correctly.

A small example of what mypy prevents:

```
def add(a: int, b: int) -> int:
    return a + b

add("100", 5)  # mypy: Argument 1 to "add" has incompatible type "str"; expected "int"
```

Result: `ruff check .` . All checks passed, `ruff format --check .` files already formatted, mypy
research/helios Success: no issues found.

6. Step 019: automatic checks before every commit

What a git hook is

A hook is a script git runs automatically at a certain moment. The **pre-commit hook** runs right before a commit is saved. If any check fails, the commit is stopped. The tool **pre-commit** manages this hook for us: we list the checks in one file and run `uv run pre-commit install` once, which writes the hook into `.git/hooks/pre-commit`.

Figure 2: what happens on git commit

```
you type: git commit -m "..."  
          |  
          v  
git runs .git/hooks/pre-commit  
          |  
+-----+-----+-----+-----+-----+  
| fix end | trim   | check yaml | large files | ruff      | gitleaks  
| of files| spaces | check toml | (> 1 MB)   | + format | (secrets)  
+-----+-----+-----+-----+-----+  
          |  
all passed? --- yes --> commit saved  
          |  
          no --> commit STOPPED (fix, git add -A, commit again)
```

The file `.pre-commit-config.yaml`

`.pre-commit-config.yaml`, copied from the project

```
# HELIOS pre-commit hooks (step 019)  
# These checks run automatically before every commit made from Ubuntu / VS Code "WSL: Ubuntu".  
# Install once per laptop: uv run pre-commit install  
# Run on everything now:   uv run pre-commit run --all-files  
# gitleaks must be installed in Ubuntu (~/.local/bin/gitleaks, official release binary).  
  
# Never auto-edit the source documents (SRS spec, reports)  
exclude: ^docs/(spec|reports)/  
  
repos:  
- repo: https://github.com/pre-commit/pre-commit-hooks  
  rev: v6.0.0  
  hooks:  
    - id: end-of-file-fixer  
    - id: trailing-whitespace  
      args: [--markdown-linebreak-ext=md]  
    - id: check-yaml  
    - id: check-toml  
    - id: check-added-large-files  
      args: [--maxkb=1024]  
  
- repo: https://github.com/astral-sh/ruff-pre-commit  
  rev: v0.16.7 # keep equal to the ruff version in uv.lock  
  hooks:  
    - id: ruff-check  
      args: [--fix]  
  
    - id: ruff-format  
  
- repo: local  
  hooks:  
    - id: gitleaks  
      name: gitleaks (block passwords and secret keys)  
      entry: gitleaks git --pre-commit --redact --staged --verbose  
      language: system  
      pass_filenames: false
```

Check	What it does	Why
end-of-file-fixer	Makes every file end with exactly one newline	Avoids noisy differences between laptops
trailing-whitespace	Removes invisible spaces at line ends	Clean diffs; Markdown line breaks are kept
check-yaml / check-toml	Stops commits with broken settings files	A broken ci.yml or pyproject.toml breaks everything
check-added-large-files (1 MB)	Blocks files larger than 1 MB	Market data must never reach GitHub
ruff-check --fix	Lint, and auto-fix simple issues	Same rules as step 018
ruff-format	Formats code	Same style for everyone
gitleaks	Scans the staged changes for passwords, keys, tokens	A leaked key is a real security incident
exclude: ^docs/(spec reports)/	Skips the SRS spec and reports	Checks must never rewrite our source documents
rev: v0.16.7	Hook version of ruff	Kept equal to the ruff in uv.lock, so local, hook and CI agree

gitleaks: the secret blocker

gitleaks looks for patterns of real secrets (for example GitHub tokens starting with ghp_, AWS keys, private keys). We installed the official gitleaks program version 8.30.1 (7.85 MB) from its GitHub release page instead of building it from source, which would have needed a 70 MB Go download on our slow internet.

Before installing, we compared the file's **SHA-256 checksum** with the official checksum list published with the release. The result was OK, which proves the file was not damaged or tampered with during download. The program lives at ~/.local/bin/gitleaks in Ubuntu.

How we tested it without committing

We never commit on the team's behalf, so we tested differently. We ran every hook directly on all project files (8 of 8 passed), then created a file with a **fake** GitHub token in Ubuntu's /tmp folder, outside the project, and scanned it:

```
RuleID:  github-pat      <- gitleaks recognised the fake token
File:    /tmp/helios_fake_secret_test/config.py
gitleaks exit code: 1    <- 1 means a commit containing it would be BLOCKED
project scan: no secrets found
```

Why commits must be made from Ubuntu

The hook script remembers where pre-commit was installed: inside Ubuntu. When a commit is made from Git Bash or PowerShell on Windows, the hook cannot find pre-commit and stops the commit. You saw exactly this message once:

```
`pre-commit` not found. Did you forget to activate your virtualenv?
```

Rule: This is on purpose (we chose it). It guarantees every commit is checked. Commit from the Ubuntu terminal or the VS Code window showing WSL: Ubuntu. Pushing can still be done from Git Bash, because pushing does not run the commit hooks.

7. Step 020: GitHub CI (Continuous Integration)

What CI is

CI means a server checks the code automatically every time someone pushes. GitHub's own service for this is **GitHub Actions**. For each push to main and each pull request, GitHub starts a brand-new Ubuntu machine, downloads our code, installs exactly the libraries from uv.lock, runs ruff, mypy and pytest, and shows a **green check** or a **red cross** next to the commit.

Why we need it even though checks already run on our laptop

- **Clean machine:** a laptop may have leftovers that hide a problem. CI starts from nothing every time, so it proves the project works for anyone who clones it.
- **Nobody can skip it:** hooks can be bypassed on a laptop; CI runs on GitHub no matter what.
- **Team safety:** once main is protected, a pull request can only be merged when CI is green, so broken code never reaches main.
- **Evidence:** the green history on GitHub shows the project was checked at every step, which helps in reviews and the final report.

The file .github/workflows/ci.yml

.github/workflows/ci.yml, copied from the project

```
# HELIOS continuous integration (step 020)
# GitHub runs these checks on a clean Linux machine for every push to main
# and for every pull request. A red X means something must be fixed before merging.
name: CI

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  checks:
    name: lint, types, tests
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - name: Check out code
        uses: actions/checkout@v7
```

```

- name: Install uv (same version as the team laptops)
  uses: astral-sh/setup-uv@v10.1.0 # setup-uv publishes exact tags only (no moving "v10"
tag)
  with:
    version: "0.12.13"
    enable-cache: true

- name: Install Python 3.12 and exact dependencies from uv.lock
  run: uv sync --locked

- name: Ruff lint
  run: uv run ruff check .

- name: Ruff format check
  run: uv run ruff format --check .

- name: Mypy (strict)
  run: uv run mypy research/helios

- name: Pytest
  run: uv run pytest -v

```

Line or block	Meaning
name: CI	The workflow's name shown on GitHub.
on: push: branches: [main] and pull_request:	When it runs: every push to main and every pull request.
permissions: contents: read	CI may only read the code, never change it. Safer.
concurrency ... cancel-in-progress: true	If you push twice quickly, the older run is cancelled to save time.
jobs: checks: name lint, types, tests	One job with the name you see on GitHub.
runs-on: ubuntu-latest	GitHub gives a fresh Ubuntu machine for every run.
timeout-minutes: 15	A stuck run is stopped automatically.
uses: actions/checkout@v7	Downloads our repository onto that machine.
uses: astral-sh/setup-uv@v10.1.0 with version: "0.12.13"	Installs uv, the same version as our laptop; enable-cache makes later runs faster.
run: uv sync --locked	Installs Python 3.12 and exactly the versions in uv.lock. --locked fails if uv.lock is out of date.
run: uv run ruff check ./ruff format --check .	Lint and formatting must pass.
run: uv run mypy research/helios	Strict type check must pass.
run: uv run pytest -v	All tests must pass.

The first run failed, and how we fixed it

```
Error: Unable to resolve action `astral-sh/setup-uv@v10`, unable to find version `v10`
```

Many GitHub Actions publish a short moving tag like v7 that always points to the newest 7.x version. actions/checkout does this, so @v7 works. The uv action does **not**: it only publishes exact tags such as v10.1.0. We checked the real tags through GitHub's API and changed the line to astral-sh/setup-uv@v10.1.0. The next push was **green**.

Note: In CI there is no .env file and no database, so the database test from step 025 is **skipped** there on purpose. Locally, with Docker running, it runs and passes.

8. Steps 021 and 022: the team

Step 021: docs/TEAM.md

A rulebook so all three members work the same way. Its ten sections:

#	Section	Main points
1	Team and roles	Q (quant research) fixed: Abhishek. S (systems) and M (ML): shared by all three, names chosen weekly.
2	Module owners	Every 5th-semester module has one owner for the current week.
3	Where and how to work	Work in Ubuntu; commit only from Ubuntu or VS Code WSL; venv outside synced folders; always <code>uv sync</code> and <code>uv add</code> .
4	Branches	main is protected; branch name = task ID + description, e.g. 5B-05-timestamps.
5	Commit messages	Start with the task ID, e.g. 5B-05: normalise timestamps.
6	Pull request rules	CI green, one reviewer who did not write the code, a test, no data or secrets, step ticked.
7	Never commit	data/, .env and secrets, venv and caches, model files, hand-typed results.
8	Weekly routine	Monday: pick steps and names. Friday: show what was merged and tick boxes.
9	Research integrity	No invented results; keep rejected alphas; paper trading only; report failures.
10	New laptop checklist	Steps 001 to 019 condensed so Anuj and Indra can set up their laptops.

Step 022: every member runs the project

Laptop	Status
Abhishek	Done: <code>uv sync</code> --locked worked, pytest passed. GitHub has every file a fresh clone needs.
Anuj	Pending, on hold by team decision (catching up later).
Indra	Pending, on hold by team decision (catching up later).

9. Step 023: Docker and the PostgreSQL database

Why HELIOS needs a database

Later steps must store things permanently and safely: every alpha definition, every evaluation result (including rejected alphas), orders, fills, positions, P&L snapshots, risk events and an audit log. A **relational database** is built exactly for this: tables with relationships, transactions (all-or-nothing saves), and rules that stop bad data. The HELIOS specification chose **PostgreSQL**: free, reliable and used by real companies.

Why Docker

Without Docker	With Docker
Install PostgreSQL directly on Windows, with its own service, settings and uninstall problems	Nothing is installed on Windows; the database runs inside a container
Each teammate may install a different version	Everyone gets exactly postgres:16-alpine
Hard to reset if something breaks	Stop and recreate the container in seconds
The setup lives in someone's memory	The setup is a text file (docker-compose.yml) in git
CI or a server would need a separate setup	The same file works anywhere Docker runs

Image, container, volume, compose: an analogy

Docker word	Analogy	In HELIOS
Image	A sealed game disc: the program, frozen, identical for everyone	postgres:16-alpine (~111 MB download) and adminer:5 (~44 MB)
Container	The game actually running on a console	helios-postgres and helios-adminer
Volume	A memory card: your saves survive when the game is switched off	helios_helios_pgdata stores the database files
docker-compose.yml	A recipe card saying which discs to run and how	Starts both containers with one command
Port	A numbered door into the container	5432 for PostgreSQL, 8080 for Adminer

We chose the **alpine** variant of the image: still PostgreSQL 16, but built on a tiny Linux, so the download is much smaller on our slow internet. The running version is PostgreSQL 16.15.

The file `infra/docker-compose.yml`

infra/docker-compose.yml, copied from the project

```
# HELIOS local infrastructure (step 023)
# Run from the project root in Ubuntu (Docker Desktop must be running):
#   Start: docker compose --env-file .env -f infra/docker-compose.yml up -d
#   Status: docker compose --env-file .env -f infra/docker-compose.yml ps
#   Stop:  docker compose --env-file .env -f infra/docker-compose.yml down
# Database files live in the Docker volume "helios_pgdata" (not in OneDrive, not in git).
# Ports are bound to 127.0.0.1, so the database is reachable only from this laptop.
name: helios

services:
  postgres:
    image: postgres:16-alpine
    container_name: helios-postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER:?set POSTGRES_USER in .env}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in .env}
      POSTGRES_DB: ${POSTGRES_DB:?set POSTGRES_DB in .env}
    ports:
      - "127.0.0.1:${POSTGRES_PORT:-5432}:5432"
    volumes:
      - helios_pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s

      retries: 10
      start_period: 10s

  adminer:
    image: adminer:5
    container_name: helios-adminer
    restart: unless-stopped
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      ADMINER_DEFAULT_SERVER: postgres
    ports:
      - "127.0.0.1:${ADMINER_PORT:-8080}:8080"
    healthcheck:
      test: ["CMD", "php", "-r", "exit(@(fsockopen('127.0.0.1', 8080) ? 0 : 1));"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s

volumes:
  helios_pgdata:
```

Line or block	Meaning	Why
name: helios	Project name; containers and volume are grouped under it	Keeps HELIOS separate from other Docker projects
image: postgres:16-alpine	Which image to run	Fixed major version 16 for everyone
container_name: helios-postgres	A fixed readable name	Easy commands like docker logs helios-postgres
restart: unless-stopped	Starts again automatically when Docker starts	No need to start the database by hand
POSTGRES_USER: \${POSTGRES_USER:?...}	Value comes from the .env file; :? stops with an error if it is missing	No password written in the file; clear error instead of a silent empty password

Line or block	Meaning	Why
"127.0.0.1:\${POSTGRES_PORT:-5432}:5432"	Opens port 5432 only on this laptop; :- gives 5432 if not set	Nobody on the same Wi-Fi can reach the database
helios_pgdata:/var/lib/postgresql/data	Database files go into a Docker volume	Data survives restarts; it is not in OneDrive or git
healthcheck: pg_isready ...	Every 5 s Docker asks: is the database ready?	We can see healthy instead of guessing
image: adminer:5	A small web tool for databases	Look inside tables from a browser
depends_on: condition: service_healthy	Adminer starts only after PostgreSQL is healthy	Avoids connection errors at start
ADMINER_DEFAULT_SERVER: postgres	Pre-fills the server name	Inside Docker, containers find each other by service name
healthcheck: php -r ... fsockopen	Checks Adminer's web port answers	The image has PHP, so no extra tool is needed

.env and .env.example: keeping the password private

.env.example, copied from the project (the real .env is never shown)

```
# HELIOS local settings template (step 023)
# Copy this file to .env and set your own password:  cp .env.example .env
# .env is gitignored. Never commit real passwords.

# PostgreSQL (Docker container "helios-postgres")
POSTGRES_USER=helios
POSTGRES_PASSWORD=change-me
POSTGRES_DB=helios
POSTGRES_HOST=localhost
POSTGRES_PORT=5432

# Adminer web UI: http://localhost:8080
ADMINER_PORT=8080
```

File	Contains	In git?
.env.example	Names of all settings and a fake password change-me	Yes: a template so teammates know what to fill in
.env	The same settings with a random 24-character password generated on this laptop	Never: .gitignore hides it and gitleaks would block it

Security choices, all together

- **Random password** generated locally with a cryptographic random generator; it was never printed.
- **Ports bound to 127.0.0.1**, so only this laptop can connect.
- **.env ignored by git**, checked with git check-ignore.
- **gitleaks** scans every commit for leaked secrets.
- **SecretStr** in Python hides the password when settings are printed (step 025).

Result of step 023

NAME	IMAGE	STATUS	
helios-adminer	adminer:5	Up (healthy)	127.0.0.1:8080->8080/tcp
helios-postgres	postgres:16-alpine	Up (healthy)	127.0.0.1:5432->5432/tcp

```
select current_user, current_database(); -> helios | helios
adminer HTTP status: 200
```

Docker trouble we solved: Docker Desktop crashed twice with 'The file cannot be accessed by the system'. Cause: when Docker is not shut down cleanly (laptop sleeps or restarts), it leaves empty socket files in C:\Users\bit\AppData\Local\Docker\run that Windows cannot remove. Fix: quit Docker, delete those files from Ubuntu with `rm -f`, start Docker again. Prevention: quit Docker from its tray icon before shutdown or sleep. Also, the new Docker Desktop 4.90 installed itself in C:\Users\bit\AppData\Local\Programs\DockerDesktop, and the old Start-menu shortcut no longer works.

10. Step 024: looking inside the database with Adminer

Adminer is a single web page for managing databases. We opened `http://localhost:8080` and logged in. This proves the database is reachable and the password works.

Field	Value	Why this value
System	PostgreSQL	Our database type
Server	postgres	Adminer runs in Docker; inside Docker the database is reached by its service name, not localhost
Username	helios	From POSTGRES_USER
Password	(from your private .env)	Typed by you; never written in code or in this guide
Database	helios	From POSTGRES_DB

The database shows **no tables**. That is correct: tables are created in steps 057 to 062 (instruments, data snapshots, alpha definitions, alpha results and more). You will be able to watch them appear here.

11. Step 025: Python talks to the database

What we did and why

The research code must save results into PostgreSQL. So Python needs a driver that speaks PostgreSQL's network protocol, a toolkit to send SQL safely, and a way to read the connection settings from .env without ever writing the password in code. We added four libraries and wrote db.py plus tests.

Library	Version	What it does	Used for
SQLAlchemy	2.0.52	The standard Python toolkit for databases: connections, SQL, tables	All database work from now on
psycopg[binary]	3.3.5	The driver that actually talks to PostgreSQL ([binary] = ready-built, no compiler needed)	SQLAlchemy uses it underneath
Alembic	1.20.0	Creates and changes tables step by step (migrations)	From step 056
pydantic-settings	2.15.0	Reads settings from .env and environment variables with type checking	DatabaseSettings in db.py

Their helper packages came along too (greenlet, mako, markupsafe, python-dotenv). uv.lock now pins 46 packages.

The file research/helios/common/db.py

research/helios/common/db.py, copied from the project

```
"""Database connection helpers (step 025).

Settings come from environment variables or the project `.env` file (never hard-coded):
POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB, POSTGRES_HOST, POSTGRES_PORT.
The password is kept as a SecretStr, so it is hidden when settings or URLs are printed.
"""

from __future__ import annotations

from functools import lru_cache
from pathlib import Path

from pydantic import SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict
from sqlalchemy import URL, Engine, create_engine

# research/helios/common/db.py -> parents[3] is the project root (where .env lives)
PROJECT_ROOT = Path(__file__).resolve().parents[3]

class DatabaseSettings(BaseSettings):
    """PostgreSQL connection settings, read from POSTGRES_* variables."""

    model_config = SettingsConfigDict(
        env_file=PROJECT_ROOT / ".env",
        env_prefix="POSTGRES_",
```

```

        extra="ignore",
    )

    user: str
    password: SecretStr
    db: str
    host: str = "localhost"
    port: int = 5432

    def url(self) -> URL:
        """SQLAlchemy URL using the psycopg (v3) driver."""
        return URL.create(
            drivename="postgresql+psycopg",
            username=self.user,
            password=self.password.get_secret_value(),
            host=self.host,
            port=self.port,
            database=self.db,
        )

    @lru_cache(maxsize=1)
    def get_settings() -> DatabaseSettings:
        """Load settings once. Raises pydantic.ValidationError if they are missing."""
        return DatabaseSettings()

    @lru_cache(maxsize=1)
    def get_engine() -> Engine:
        """Shared SQLAlchemy engine; pool_pre_ping drops dead connections automatically."""
        return create_engine(get_settings().url(), pool_pre_ping=True)

```

Code	What it does	Why
from __future__ import annotations	Makes type hints lazy	Modern, avoids ordering problems in type hints
PROJECT_ROOT = Path(__file__).resolve().parents[3]	Finds the project folder from this file's location: common, helios, research, then the root	The .env file is found no matter which folder you run a command from
class DatabaseSettings(BaseSettings)	A settings class; pydantic-settings fills it from the environment	Typed settings with automatic checks
env_file=PROJECT_ROOT / ".env"	Read values from the project .env	No password in code
env_prefix="POSTGRES_"	Field user is read from POSTGRES_USER, and so on	Matches the names Docker uses in docker-compose.yml
extra="ignore"	Ignore other lines in .env such as ADMINER_PORT	Avoids errors for unrelated settings
password: SecretStr	The password is stored as a secret type	Printing the settings shows ***** instead of the password
host: str = "localhost" / port: int = 5432	Defaults when not set	Works out of the box on our laptop
def url(self) -> URL	Builds the connection address	One correct place to create it
drivename="postgresql+psycopg"	Database type PostgreSQL, driver psycopg version 3	Tells SQLAlchemy which driver to use
password.get_secret_value()	Unwraps the secret only at the moment of connecting	Keeps it hidden everywhere else
@lru_cache(maxsize=1)	Remembers the result after the first call	Settings are read once; one shared engine for the whole program
get_settings()	Loads settings; raises ValidationError if a required value is missing	Clear error instead of a confusing connection failure

Code	What it does	Why
<code>create_engine(..., pool_pre_ping=True)</code>	Creates the engine, a manager of reusable connections; pre-ping tests a connection before using it	If Docker restarted the database, broken connections are replaced automatically

The file `research/tests/test_db.py`

`research/tests/test_db.py`, copied from the project

```

"""Database connection tests (step 025)."""

import pytest
from pydantic import ValidationError
from sqlalchemy import text

from helios.common import db

def test_select_one() -> None:
    """Connects to PostgreSQL and runs SELECT 1. Skips when no DB settings exist (e.g. CI)."""
    try:
        db.get_settings()
    except ValidationError:
        pytest.skip("database settings not configured (no .env / POSTGRES_* variables)")
    with db.get_engine().connect() as conn:
        assert conn.execute(text("SELECT 1")).scalar_one() == 1

def test_password_is_never_shown() -> None:
    """The password must not appear when settings or the URL are printed."""
    settings = db.DatabaseSettings(
        user="u",
        password="very-secret-test-value",
        db="d",
        _env_file=None,
    )
    assert "very-secret-test-value" not in repr(settings)
    assert "very-secret-test-value" not in str(settings.url())
    assert settings.url().drivername == "postgresql+psycopg"

```

Test	What it checks	Why it is written this way
<code>test_select_one</code>	Connects to PostgreSQL and runs <code>SELECT 1</code> , which must return 1	The simplest possible proof that Python reaches the database
<code>except ValidationError: pytest.skip(...)</code>	If no database settings exist, the test is skipped, not failed	GitHub CI has no <code>.env</code> and no database, so CI stays green; locally it really runs
<code>with db.get_engine().connect() as conn</code>	Opens a connection and closes it automatically afterwards	No leaked connections
<code>text("SELECT 1") / scalar_one()</code>	Sends raw SQL and reads exactly one value	Fails loudly if the answer is not exactly one value
<code>test_password_is_never_shown</code>	Builds settings with a test password and checks it is not visible in <code>repr()</code> or <code>str(url)</code>	Guarantees the password cannot leak into logs or error messages
<code>_env_file=None</code>	This test ignores the real <code>.env</code>	The test never touches the real password

Result

```
research/tests/test_db.py::test_select_one PASSED [ 33%]
research/tests/test_db.py::test_password_is_never_shown PASSED [ 66%]
research/tests/test_smoke.py::test_smoke PASSED [100%]
===== 3 passed =====
mypy: Success: no issues found in 3 source files
ruff: All checks passed!
```

12. How a change travels

This is the full path every change now follows, from typing code to a green check on GitHub:

Figure 3: the life of a change

```
1. Edit code in the project folder (VS Code, window "WSL: Ubuntu")
2. Test locally in Ubuntu:      uv run pytest | uv run ruff check . | uv run mypy
research/helios
3. Stage the changes:          git add -A
4. Commit (from Ubuntu):       git commit -m "5X-NN: short description"
    -> pre-commit hooks run: file fixers, yaml/toml, large files, ruff, gitleaks
    -> if a hook fixed something: git add -A and commit again
5. Push (Git Bash or Ubuntu):  git push
6. GitHub Actions CI starts a fresh Ubuntu machine:
    uv sync --locked -> ruff check -> ruff format --check -> mypy -> pytest
7. Result on GitHub: green check (good) or red cross (fix and push again)
8. Later, when main is protected: open a pull request, 1 teammate reviews, merge only when green
```

Each layer catches different problems. Your laptop gives fast feedback while you work, the hooks stop bad commits and leaked secrets, and CI proves the project works on a clean machine for anyone.

13. Problems we hit and how we fixed them

Problem	Cause	Fix
Downloads very slow (38 min for libraries)	Slow internet connection	Ran downloads in the background; uv caches them so it happens once
ruff wanted to rewrite a Markdown document	ruff 0.16 formats Python examples inside .md files	extend-exclude *.md in pyproject.toml
Commit from Git Bash failed: pre-commit not found	Hooks were installed in Ubuntu (our choice)	Commit from the Ubuntu terminal; push from Git Bash is fine
CI failed: unable to find setup-uv v10	setup-uv publishes exact tags only	Changed to astral-sh/setup-uv@v10.1.0; checked tags via GitHub API
Docker Desktop crashed at start	Leftover socket files after an unclean shutdown	Quit Docker, delete the leftovers from Ubuntu, restart; quit Docker before sleep
Docker Start-menu shortcut broken	Docker 4.90 moved to a per-user install folder	Start it from C:\Users\bit\AppData\Local\Programs\DockerDesktop
Strange output such as a mangled user name	PowerShell changes \$ variables and quotes before Ubuntu sees them	Run checks as small script files inside Ubuntu
Old spec file tab shown in red in VS Code	The file was moved to docs/spec	Close the tab without saving; open it from docs/spec

14. Cheat sheet

Run these in the **Ubuntu terminal** after `cd /mnt/c/Users/bit/OneDrive/Desktop/helios`.

Command	What it does
<code>uv sync</code>	Installs exactly what <code>uv.lock</code> says (after pulling new changes)
<code>uv add <library></code>	Adds a new library and updates <code>pyproject.toml</code> and <code>uv.lock</code>
<code>uv run pytest -v</code>	Runs all tests
<code>uv run ruff check .</code>	Looks for mistakes
<code>uv run ruff format .</code>	Formats all Python code
<code>uv run mypy research/helios</code>	Strict type check
<code>uv run pre-commit run --all-files</code>	Runs all commit checks without committing
<code>git status</code>	Shows what changed
<code>git add -A</code> then <code>git commit -m "..."</code>	Stages and commits (hooks run)
<code>git push</code>	Uploads commits to GitHub (CI runs)
<code>docker compose --env-file .env -f infra/docker-compose.yml up -d</code>	Starts PostgreSQL and Adminer
<code>docker compose --env-file .env -f infra/docker-compose.yml ps</code>	Shows container status (should say healthy)
<code>docker compose --env-file .env -f infra/docker-compose.yml down</code>	Stops the containers (data is kept in the volume)
<code>docker logs helios-postgres --tail 40</code>	Shows recent database log lines

Remember: Open Docker Desktop before using any docker command, and quit it from the tray icon before you shut down or put the laptop to sleep. Adminer is at <http://localhost:8080>.

15. Rules we follow, and what comes next

Rules

- **One project folder:** `C:\Users\bit\OneDrive\Desktop\helios`. No second copies.
- **Commit from Ubuntu** (hooks live there). The team commits; the assistant never commits or pushes.
- **Never commit:** `data/`, `.env` or any secret, the `venv`, caches, large generated files.
- **Every step has a test** and a clear 'done when' check.
- **No invented results:** every number must come from a real run.

Status after step 025

Sprint	Steps	Status
1: Laptop setup	001 to 009	Done
2: Project foundation	010 to 022	Done, except team parts on hold: 010 (invite teammates, protect main) and 022 (teammates' laptops)
3: Database and config	023 to 025	Done

Next: step 026, the config loader

Step 026 creates `research/helios/common/config.py` with `load_config(path, Model)`. It will read a YAML settings file and check every value with `pydantic`, so a threshold written as text instead of a number gives a clear error instead of silently wrong results. The selection gates ($\text{Sharpe} > 1$, $\text{Fitness} > 1$, $1\% < \text{Turnover} < 70\%$), trading fees, data split dates and the list of coins will all be loaded this way. A test will check that a correct file loads and a broken one is rejected.